Ray Tracer

Juan Francisco Luna Posso

Computer Graphics

Universidad Panamericana

June 2026

Abstract

This report documents the full development of a ray tracer built from scratch in Java. The project went through 8 versions, starting from basic ray-sphere intersection all the way to a physically-based renderer with Blinn-Phong shading, multiple light types, shadows, reflection, refraction , BVH acceleration, and multithreading. Three final renders are presented, each one with a personal story behind it.

Contents

Introduction

When I started this project I honestly did not know how much math was going to be involved. I thought: *"how hard can it be, you just shoot rays and see what they hit"*. Spoiler: it was a lot harder than that. But that is also what made it interesting.

The idea behind a ray tracer is simple: for every pixel on the screen, you shoot a ray from the camera into the scene and ask what does it hit, then you compute the color based on the materials and lights. The hard part is making all of that physically correct, fast, and actually looking good.

This report explains how I built the ray tracer step by step, what each version added, what the three final scenes mean to me, and honestly where I think I did well and where I think I could have done better.

Development Process

The ray tracer was built in 8 iterations. Each version had a specific goal. This section walks through each one explaining what was added and why.

v0.1 — The Very Beginning: Rays and Spheres

The first version was basically just proving that the math works. I implemented:

- A `Ray` class with an origin and a direction.
- A `Camera` that generates one ray per pixel using perspective projection.
- A `Sphere` with the quadratic ray-sphere intersection formula.
- Flat shading with a single hardcoded color.

The intersection test for a sphere of center $\mathbf{c}$ and radius r is derived from $|\mathbf{o}+t\mathbf{d}-\mathbf{c}|^2 = r^2$, which expands to a quadratic $at^2 + bt + c = 0$. If the discriminant is positive, the ray hits the sphere.

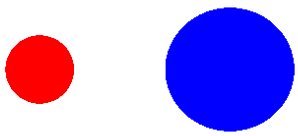

Figure 1: v0.1 — First image ever. Three spheres, flat colors, no lighting. It worked.

At this point it was basically just coloring spheres. But the perspective projection was already correct and the intersection math was solid, so this was the foundation for everything else.

v0.2 — Triangle Meshes

Spheres are fine but they are limited. Real 3D models are made of triangles. I added the `Triangle` class using the **Möller-Trumbore algorithm** for ray-triangle intersection. This algorithm computes the hit using barycentric coordinates (u, v, w) where $w = 1 - u - v$, which I would later use for normal interpolation.

The math is:

$$\begin{pmatrix} t \\ u \\ v \end{pmatrix} = \frac{1}{\mathbf{d} \cdot (\mathbf{e_1} \times \mathbf{e_2})} \begin{pmatrix} (\mathbf{o} - \mathbf{v_0}) \cdot (\mathbf{e_1} \times \mathbf{e_2}) \\ \mathbf{d} \cdot ((\mathbf{o} - \mathbf{v_0}) \times \mathbf{e_2}) \\ \mathbf{d} \cdot (\mathbf{e_1} \times (\mathbf{o} - \mathbf{v_0})) \end{pmatrix} \tag{1}$$

where $\mathbf{e_1} = \mathbf{v_1} - \mathbf{v_0}$ and $\mathbf{e_2} = \mathbf{v_2} - \mathbf{v_0}$.

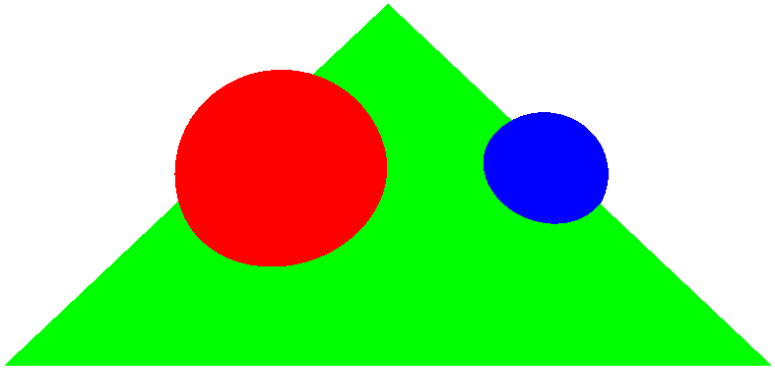

Figure 2: v0.2 — Triangle mesh rendering. The object looks blocky because normals are not interpolated yet.

v0.3 — OBJ File Loading

Hardcoding vertices is not practical for real models. I built the `ObjReader` class that parses `.obj` files. It reads:

- `v` — vertex positions.
- `vn` — vertex normals.
- `f` — faces (triangles or quads, which get split into two triangles).

After loading, every model is automatically normalized to fit inside a unit cube centered at the origin, then scaled and translated to the desired position. This made it easy to drop any model into a scene without worrying about its original coordinate system.

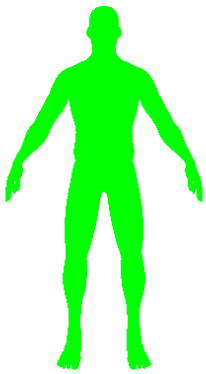

Figure 3: v0.3 — OBJ model loaded from a file. The Inca warrior mesh. Still no shading.

v0.4 — Basic Lighting (Lambertian Diffuse)

A ray tracer without lighting is just silhouettes. In this version I added the `Light`, `DirectionalLight`, and `PointLight` classes, and implemented basic Lambertian (diffuse) shading:

$$I_{\text{diffuse}} = k_d \cdot C_{\text{object}} \cdot C_{\text{light}} \cdot L_I \cdot \max(0,\ \hat{N} \cdot \hat{L}) \tag{2}$$

where $\hat{N}$ is the surface normal, $\hat{L}$ is the direction toward the light, and L_I is the light intensity. For point lights, $L_I = I/d^2$ (inverse-square falloff).

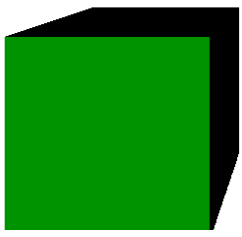

Figure 4: v0.4 — Diffuse shading. The model finally has volume and depth.

v0.5 — Phong Normal Interpolation (Smooth Shading)

Hard flat shading makes every triangle visible because each triangle has a single normal. I added **Phong normal interpolation**: instead of using one normal per triangle, I interpolate the per-vertex normals using the barycentric coordinates from the intersection:

$$\hat{N}_{\text{interp}} = \text{normalize}\big(w \cdot \hat{N}_0 + u \cdot \hat{N}_1 + v \cdot \hat{N}_2 \big) \tag{3}$$

This made curved surfaces look smooth even with a low polygon count.

Figure 5: v0.5 — Same mesh as before, now with normal interpolation. Much smoother.

v0.6 — Multiple Light Types

I refactored the lighting system into a proper OOP hierarchy. The abstract `Light` class defines the interface, and two concrete classes implement it:

- **DirectionalLight** — parallel rays everywhere, like the sun. No falloff, infinite distance.
- **PointLight** — rays emanate from a point in all directions. Inverse-square falloff: $L_I = I/d^2$.

This made adding multiple lights to a scene trivial. The renderer just loops over all lights and accumulates their contributions.

v0.7 — Shadows and Light Falloff

Without shadows everything looks flat and fake. I added hard shadow testing by casting a *shadow ray* from the hit point toward each light:

1. Compute the hit point $\mathbf{p} = \mathbf{o} + t\mathbf{d}$.
2. Offset slightly along the light direction to avoid self-intersection: $\mathbf{p}' = \mathbf{p} + 0.001\hat{L}$.
3. Cast a ray from $\mathbf{p}'$ toward the light.

4. If the ray hits anything before reaching the light source, that hit point is in shadow and that light's contribution is skipped.

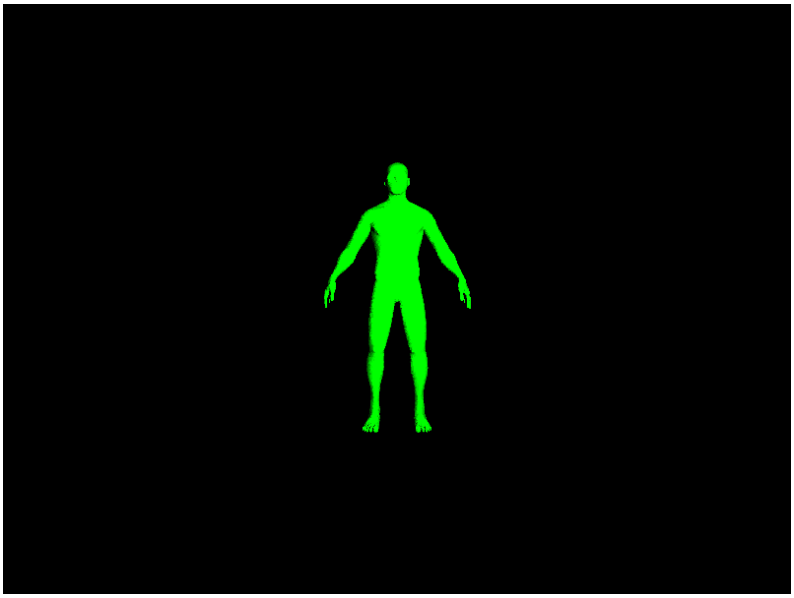

Figure 6: v0.7 — Hard shadows and multiple lights. The scene finally looks grounded.

v1.0 — Blinn-Phong, Reflection, Refraction, and BVH

This was the biggest update. Four major features were added:

Blinn-Phong Shading. The complete shading model is:

$$I = k_a C_o + \sum_{\text{lights}} \left[k_d C_o C_l L_I (N \cdot L) + k_s C_l L_I (N \cdot H)^{\alpha} \right] \tag{4}$$

where $H = \text{normalize}(L + V)$ is the half-vector between light and view direction, and α is the shininess exponent. The half-vector makes specular highlights more physically correct than the pure Phong model.

Reflection. A reflected ray is computed as:

$$\mathbf{R} = \mathbf{D} - 2(\mathbf{D} \cdot \hat{N})\hat{N} \tag{5}$$

The reflected color is blended with the direct shading by the material's reflectivity coefficient. The recursion depth is limited to `MAX_DEPTH` $= 4$ bounces.

Refraction (Snell's Law). For transparent materials:

$$n_1 \sin\theta_1 = n_2 \sin\theta_2 \tag{6}$$

The refracted direction is:

$$\mathbf{T} = \frac{n_1}{n_2}\mathbf{D} + \left(\frac{n_1}{n_2} \cos\theta_i - \cos\theta_t \right) \hat{N} \tag{7}$$

When the ray exits an object (detected by $\mathbf{D} \cdot \hat{N} > 0$), the normal is flipped and the IOR values are swapped. Total internal reflection is handled correctly when $\sin^2\theta_t > 1$.

BVH Acceleration + Multithreading. With complex meshes, brute-force intersection tests are too slow. I built a Bounding Volume Hierarchy (BVH) using axis-aligned bounding boxes (AABBs) and a median-split strategy. Rendering was also parallelized using Java's `ExecutorService` with one thread per row.

Figure 7: v1.0 — Firts steps of the Yahuarcocha scene

Final Renders

The three final renders are each 4096×2160 pixels Each one tells a story.

Scene 1 — The Silent Game

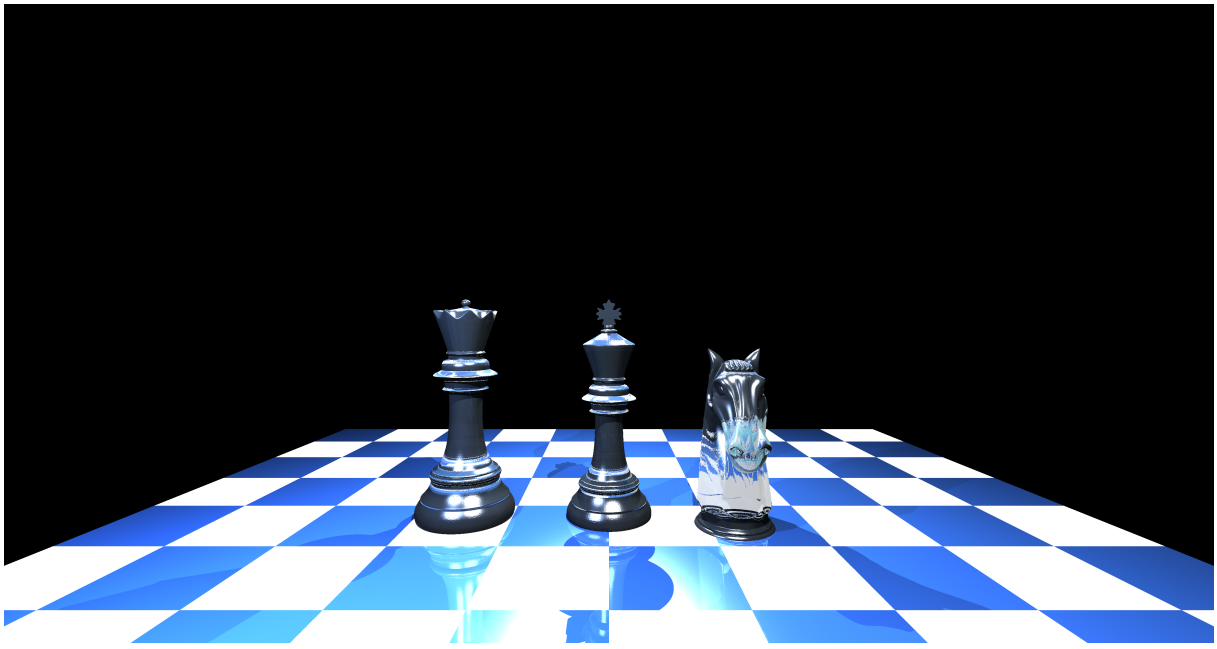

Figure 8: Scene 1: The Silent Game. Chess pieces made of glass on a reflective board.

The Story. I like to play chess because it forces you to think ahead. Not just with one part of yourself, but with everything. Chess teaches you that you have to be strong both in your mind and in your body — a tired mind does not play well. This scene represents that.

The chess pieces are made of glass. Glass is fragile but also transparent, like an honest player who has nothing to hide. The board reflects the pieces, because every move you make leaves a mark on the game. The five lights illuminate from different angles so there are no perfect shadows — just like in a real game, every position has a weakness somewhere.

Technical details. The pieces (queen, king, knight) are loaded from OBJ files. The board is a 10×10 grid of triangles alternating between two materials: one dark and very reflective (water, IOR 1.33), one white and semi-reflective. The glass pieces use IOR 1.5 with 85% transparency and 20% reflectivity. There are five lights: one directional and four point lights at different heights and positions to create complex shadow patterns across the board.

Scene 2 — Street Level

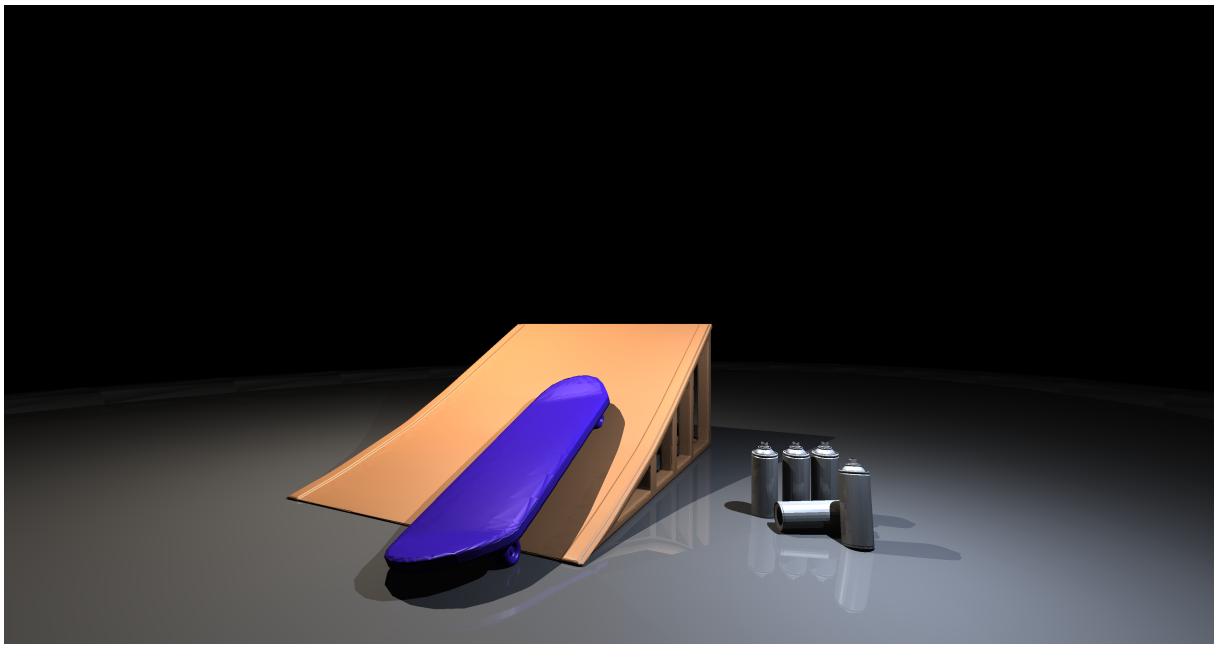

Figure 9: Scene 2: Street Level. A skate park with a ramp, a board, and a spray can.

The Story. I have always been attracted to the urban style. Skate culture, street art, the energy of the city — there is something honest about it. Nobody is pretending to be something they are not. You fall, you get up, you try the trick again. This scene is my tribute to that world.

The spray can sitting on the ramp represents street art, which for me is one of the most creative expressions that exists. It is art that belongs to everyone because it is in the street, not in a gallery. The skateboard leaning at an angle is mid-motion, frozen in the moment before the trick. The floor has a slight reflection, like wet concrete after rain.

Technical details. Four OBJ models: a concrete floor plane (`suelo.obj`), a halfpipe ramp (`rampa.obj`), a skateboard (`patineta.obj`), and a spray can (`sprycan.obj`). The floor material uses 15% reflectivity to simulate slightly wet concrete. Lighting uses a directional light for ambient daylight and two point lights — one warm (255, 240, 220) and one cool (200, 220, 255) — for contrast.

Scene 3 — Yahuarcocha: Lake of Blood

Figure 10: Scene 3: Yahuarcocha. An Inca warrior stands over the crimson lake.

The Story. Yahuarcocha is a lake located in Ibarra, Ecuador. Its name comes from Quichua and it means *Lake of Blood*. The legend behind the name is one of the darkest moments in Andean history: in 1486, the Inca army of Huayna Cápac massacred more than 30,000 Caranqui warriors after a long resistance. The bodies were thrown into the lake, turning the water red. That is how the lake got its name.

This scene shows an Inca warrior standing at the edge of the blood-red lake. At his feet, the skulls of the fallen. The warrior is not celebrating — he is witnessing. The warm golden light from the right side represents the sun of the Inca empire. The cold blue light from the left represents the death and sorrow of the Caranqui people. The crimson water reflects nothing, because some things should not be forgotten.

I chose this scene because it connects me to the history of Ecuador, where this lake still exists today and people go to enjoy boat rides without knowing — or maybe knowing and choosing not to talk about — what happened there centuries ago. History should be remembered.

Technical details. Two OBJ models: `inca.obj` (the warrior figure) and `skulls.obj` (scattered at the base). The lake is two large triangles forming a plane with a dark red material (RGB 80, 5, 5) with 50% reflectivity — it has a dark mirror-like surface. Lighting uses one directional light for overall fill and two point lights: one warm-orange

at 800 intensity (the Inca sun) and one cool-blue at low intensity (the cold grief). Camera FOV is 75°.

Greatest Challenges

Refraction and Normal Direction

By far the hardest part was getting refraction to work correctly. When a ray exits a transparent object, the normal points outward but the ray is going in the opposite direction. I kept getting rays that went the wrong way or got stuck inside objects forever.

The fix was to detect which side of the surface the ray is on by checking $\mathbf{D} \cdot \hat{N} > 0$. If positive, the ray is exiting, so I flip the normal and swap the IOR values. It sounds simple but it took a lot of debugging to get right.

Shadow Acne

Before I added the small offset (`0.001`) to shadow rays, almost every surface looked like it had black dots all over it. This happens because the shadow ray starts exactly at the surface, which due to floating-point precision sometimes intersects the same surface it just hit. The tiny offset steps the shadow ray origin slightly away from the surface along the light direction, which solved the problem.

BVH Construction

The Bounding Volume Hierarchy was conceptually clear but tricky to implement. Getting the median split correct and making sure the AABB slab intersection test handled edge cases (ray parallel to a slab, degenerate triangles) required several iterations. But the performance gain was worth it — without BVH, complex scenes with thousands of triangles would take minutes instead of seconds.

Multithreading and BufferedImage

Java's `BufferedImage` is not thread-safe for concurrent pixel writes. I had to make sure that each thread writes to a different row (no two threads ever write to the same pixel), which is naturally guaranteed by the row-per-thread parallelization. This gave a near-linear speedup with the number of CPU cores.

OBJ Model Placement

Getting models into the right position and orientation was more painful than expected. Each OBJ file comes with its own coordinate system. Some models are Y-up, some Z-up, some are huge and some are tiny. I built a normalization pipeline that scales every model to a unit cube, then applies user-defined scale, rotation, and offset. Even with that, fine-tuning the rotation angles required a lot of trial and error.

Self-Evaluation

Table 1: Self-evaluation against grading criteria

Criterion	Points	Justification
Ray Tracer v1.0 (10%)		
Shadows	5/5	Shadow rays with BVH early exit and self-intersection offset. Works correctly for both directional and point lights.
Blinn-Phong (specular + shininess)	5/5	Full Blinn-Phong implementation with ambient, diffuse, and specular components. Shininess is configurable per material.
Reflection (25%)		
Reflected objects shown correctly	15/15	Reflection rays are computed using the standard reflection formula and blended according to material reflectivity.
At least 2 bounces	10/10	Recursive reflections are supported with `MAX_DEPTH = 4`.
Refraction (25%)		
Refracted objects shown correctly	15/15	Transparent materials correctly transmit and bend light through the object.
Light changes direction correctly	10/10	Snell's law is implemented with support for entry/exit transitions and total internal reflection.
Coding Principles (10%)		
No unnecessary code	1/1	The codebase was cleaned during development and no major unused components remain.
Proper OOP	2/2	Responsibilities are separated across `Object3D`, `Light`, `Material`, `Scene`, and `Camera`.
Reusable	1/1	Materials, lights, and object loaders can be reused across different scenes.
Flexible	1/1	New scenes can be configured with relatively small modifications.
No bugs	1.5/2	No major bugs were observed during testing, although minor numerical precision limitations are inherent to ray tracing.
Scalable	0.5/1	BVH and multithreading improve performance, although no formal benchmarking was conducted.
Comments	0.5/1	Most complex sections are documented, although some utility methods could benefit from additional comments.
Code is neat	1/1	Consistent naming, formatting, and class organization throughout the project.
Total	**92/100**	

Table 2: Final Render Evaluation

Criterion	Points	Justification
3 Complex Final Renders (10%)		
Each render has a story	3/3	Every scene was designed around a personal theme and narrative.
4096 × 2160 PNG	2/2	All final renders were generated at 4K UHD resolution.
No spheres or teapots	2/2	Final scenes are based on OBJ models rather than primitive showcase objects.
Aesthetic	2.5/3	The scenes achieve the intended visual result, although anti-aliasing and environment lighting would further improve image quality.
Total	**92/100**	

To be honest, the main thing missing that would improve the visual quality is anti-aliasing. Right now each pixel casts exactly one ray, which means diagonal edges look jagged. Super-sampling,casting multiple rays per pixel and averaging, would fix this. I also think the sky/background being pure black makes the scenes feel a bit empty. A gradient or environment map would make a big difference aesthetically.

But overall I am proud of what this project became. It started as a sphere on a screen and ended as a physically-based ray tracer with Snell's law, BVH trees, and 4K renders. That is a long way to go in one semester.

Conclusion

Building a ray tracer from scratch was one of the most complete programming experiences I have had. Every component depends on the previous one: you cannot have shadows without intersection tests, you cannot have refraction without knowing which side of the surface you are on, you cannot render complex models in reasonable time without an acceleration structure.

The three final images each mean something to me. The chess board is about discipline and strategy. The skate park is about the urban culture I grew up with. Yahuarcocha is about history that should not be forgotten. That is what makes this project feel real to me, it is not just math and code, it is something I actually put thought into.

The code is available at the project repository and all renders are included as PNG files at 4096 × 2160 resolution.